

Module 2 Assignment: Group Project

Overview

- **Team Size:** 2 to 3 learners per team
- **Estimated Time:** Roughly 5 to 8 hours per team member outside class
- **Submission:** GitHub repository link, one per team
- **Presentation:** Lesson 2.19

Why a Group Project

Instead of everyone building the same app, your team chooses its own app idea and applies the concepts covered across the React lessons to a domain of your choosing.

This is a chance to:

- Work in a small team and make your own product and technical decisions
- Produce something you can talk about in an interview or portfolio
- Practise applying React concepts to a problem no one has handed you a spec for

What This Project Reinforces

- Component-based UI with JSX, props, and composition
- State with `useState` , and `useReducer` where it fits
- Handling events and lifting state up
- Conditional rendering and rendering lists
- Fetching and displaying data with `useEffect`
- Context API, where the app's complexity warrants it
- Client-side routing with React Router
- Working as a team on a shared codebase with Git and GitHub

Team Formation

- Teams of 2 to 3, announced separately by your instructor
- Agree on roles or ownership early, but make sure **every member** touches:
 - State management
 - At least one route
 - At least one data-fetching or component-composition task
- Use a shared GitHub repo with regular commits from all members
 - Commit history is part of how individual contribution is assessed

Choosing Your App Idea

Pick an idea your team can realistically finish in the time available. Some starting points:

- A to-do or productivity app
- A recipe or meal-planning app
- A personal budgeting or expense tracker
- A movie, book, or game discovery app backed by a public API
- A weather or travel planning app
- An app built on open data, e.g. **data.gov.sg**

The idea does not need to be original. It needs to give your team a genuine reason to manage state, fetch or persist data, and navigate between at least two views.

Core Requirements (1/2)

- [] Built with Vite
- [] Functional components and React hooks throughout, no class components
- [] UI organised into multiple, sensibly scoped components, with props passed cleanly
- [] At least two client-side routes with React Router, with navigation between them
- [] Shared state managed with `useState` / `useReducer` ; Context only where it genuinely helps

Core Requirements (2/2)

- [] Fetches or persists data from at least one external or mock source, with loading and error handling
- [] At least one form with controlled inputs, to create a new item
- [] Displays the collection of items (Read) and lets the user delete an item
 - Editing an existing item (Update) is optional, see Bonus Challenges
- [] Deployed to a public URL (Vercel, Netlify, or GitHub Pages)

Data Sources

Depending on what your app idea needs:

- **MockAPI**: a hosted mock backend, no server to run yourself
- A **free public API** relevant to your app idea
- Open datasets from data.gov.sg or similar
- `localStorage` for client-side persistence

Not sure which public API to use? Browse [Public APIs](#) (GitHub), [PublicAPIs.dev](#), or [RapidAPI Hub](#).

Check rate limits, API keys, and free-tier limits before committing to one.

Styling: Optional and Not Graded

This module has not covered CSS, Tailwind, or component libraries in depth, so the project focuses on **functionality**: component structure, state, routing, and data fetching.

Want a more polished look anyway? You are welcome to:

- Write your own CSS
- Reuse or adapt styles from the CRM base project or other lesson code
- Try Tailwind CSS or a component library

Entirely optional, will not affect your grade either way.

Team Collaboration

- [] All team members have commits in the repository's history
- [] Feature branches and pull requests, or an equivalent agreed workflow, not everyone pushing straight to `main`
- [] Basic project management is visible: a GitHub Projects board, issues, or a shared task list showing how work was divided

Bonus Challenges: Easy & Medium

Easy

- Search or sort on a list view
- Loading skeletons or spinners for async data
- Responsive down to mobile screen widths

Medium

- Let the user edit an existing item (Update)
- Mock authentication flow
- Automated tests with React Testing Library
- Optimistic UI updates for create/update/delete

Bonus Challenges: Hard

- Add **drag-and-drop** reordering of items in a list, using native browser drag events (no library)

Timeline

Milestone	When
Project briefed	End of final React web lesson
Team and app idea confirmed	Shortly after briefing
Progress check-in	During a coaching session
Final submission	Before Lesson 2.19
Presentation and slide deck due	Lesson 2.19

Confirm your app idea and data source with your instructor early, so scoping concerns can be caught before significant work is done.

A Note on Time

The calendar window is longer than the estimated hours of work, since other lessons and coaching sessions run in parallel.

Copilot and other AI tools speed up writing individual pieces of code. They do **not** remove the time needed to:

- Agree on your app idea and data shape as a team
- Integrate each member's work
- Debug issues that come up once your pieces are combined

Do not leave this to the last few days.

Project Presentation

At Lesson 2.19, every team presents the technical aspects of their project with a live demo.

Each team prepares a short slide deck covering:

- **Problem statement:** what the app does and who it is for
- **Technologies used:** stack, libraries, and data source, and why
- **Screenshots** of the working application
- **Learnings** of each team member
- **Challenges encountered** during the project

Presentation Format

10 to 15 minutes per team, including the live demo.

- Keep slides brief
- This is a presentation with a demo, not a design document
- Favour screenshots and a working demo over dense text

Submission Checklist

Each team submits one GitHub repository link via NTU Blackboard, with a `README.md` that includes:

- ☐ App name and a short description of what it does and who it is for
- ☐ Team member names and a note on how work was divided
- ☐ How to install and run the project locally
- ☐ The deployed URL
- ☐ A screenshot or short recording of the main features working
- ☐ A note on any bonus challenges completed
- ☐ The AI and Tools disclosure

AI and Tools

You are provided with Copilot credits for this project. You may use AI assistants (Claude, Copilot, ChatGPT) to help scaffold code, debug errors, or review pull requests.

Every team member must be able to explain any part of the codebase they are asked about.

In your `README.md`, briefly note:

- Which AI tools you used, and for what
- Any code adapted from external sources or tutorials, with a link

Grading

The project is graded as a **single team grade**, shared by all members.

Commit history and the collaboration requirements shown earlier are how individual contribution is assessed; make sure every member's work is visible in the repository.

Resources

- [React docs: Thinking in React](#)
- [React docs: Managing State](#)
- [React docs: Passing Data Deeply with Context](#)
- [React Router docs](#)
- [Vite docs](#)
- [MockAPI](#)
- [data.gov.sg](#)
- [Public APIs \(GitHub list\)](#) / [PublicAPIs.dev](#)

Questions?

Confirm your team and app idea with your instructor soon.